

Team 4: Presentation 3

Ben Pilnick, Isabella Toth, Akshay Anand, and Zoe Lively

Zoe/Ben - Mandatory Changes Results

	Entropy Regularization	Gated Attention	ER/GA	No Dropout W/o AA, AA	ER/GA/~DO W/O AA, AA
W/o Attention Analysis	Test Loss: 0.7187 Test Acc: 0.6842 Samples: 19 782482	Test Loss: 0.6899 Test Acc: 0.7895 Samples: 19 782750	Test Loss: 0.8888 Test Acc: 0.8421 Samples: 19 782596	Test Loss: 0.3294 Test Acc: 0.8421 Samples: 19 MIL_train_783532	Test Loss: 0.3956 Test Acc: 0.9474 Samples: 19 MIL_train_783730
With Attention Analysis	Test Loss: 0.7187 Test Acc: 0.6842 Samples: 19 782575	Best Test Loss: 0.3181 Test Acc: 0.8421 Samples: 19 782783	Test Loss: 0.3507 Test Acc: 0.8421 Samples: 19 782745	Test Loss: 1.0805 Test Acc: 0.7895 Samples: 19 MIL_train_782872	Test Loss: 1.0805 Test Acc: 0.7895 Samples: 19 MIL_train_782849

Zoe - Option 2

Objective:

Execute the code on multiple splits such that all benign splits are tested. Note that for cases that have been split into multiple benign cases, either all of them must be there in the test data, or none of them must be in test data, to avoid data leakage. For example, either cases 22, 107, 108 should be together in the test data, or none of them must be in the test data. Report the 1) plot of val & train loss vs epoch, (2) confusion matrix, (3) Deviation plot, for all benign cases tested

Testing All Benign Splits

764095

827819 – with 30 epochs

- Using [ER_GA_DO_FEB27](#) with all mandatory changes (ER, GA, and DO) and new attention analysis update as new baseline
- Using presentation_3_mar3_zoe to implement new changes
 - Output in run_20260303_101951
- *Disclaimer: this version has early stopping set to 10 epochs, because my 30 epochs version is still running.*

Make_splits.py changes (prevents leakage, lets us sweep test)

- **Changed default test exclusions**

- **Old:** EXCLUDE_FROM_TEST_DEFAULT included the benign family cases (22,107,108, ...) so they never appeared in test.
- **New:** EXCLUDE_FROM_TEST_DEFAULT = {46} so benign families can be intentionally tested.

- **Added benign family JSON loader**

- **New function:** load_benign_families(path)
- **Input format:** {"benign_families": [[22,107,108], ...]}

Make_splits.py changes con't

- Added CLI arguments
 - `--benign_families_json`
 - `--test_benign_family_idx`
- Updated split function signature
 - `split_by_case_with_constraints(..., benign_families=None, test_benign_family_idx=None)`
- Modified TEST sampling logic
 - Old behavior: randomly sampled B_test benign cases from eligible benign pool.
 - New behavior: if `test_benign_family_idx` is provided, it forces the entire benign family into TEST as a block (“all-or-none”), then fills remaining benign test slots randomly.
- Why:
 - Prevents leakage across siblings (same case split into multiple IDs)
 - Enables targeted evaluation: hold out family i, train on everything else

New File: `run_benign_family_sweeps.sh` (ensures all benign splits tested)

- Loop over indices 0 1 2 3
- For each:
 - run `make_splits.py` to generate a split that forces benign family IDX into test
 - run `main.py --load_splits ...` to train/evaluate on that split
- Updated Sbatch to run Sweep instead of single `--load_splits ...`
- Why:
 - The objective requires “all benign splits are tested”
 - Keeps runs consistent and reproducible (each run corresponds to one family)

Trainer.py changes: deviation plot output

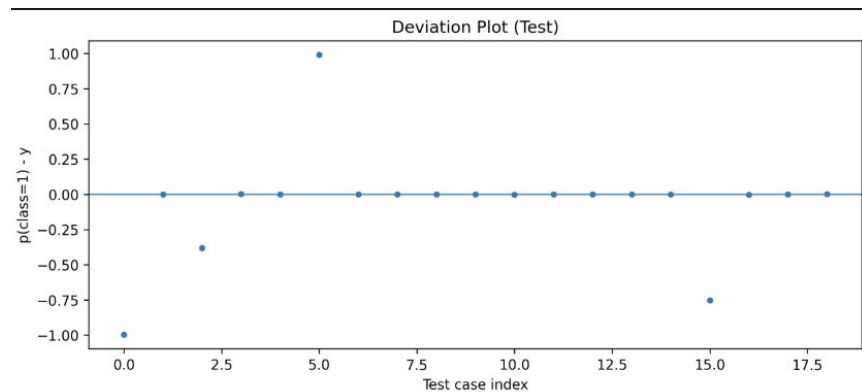
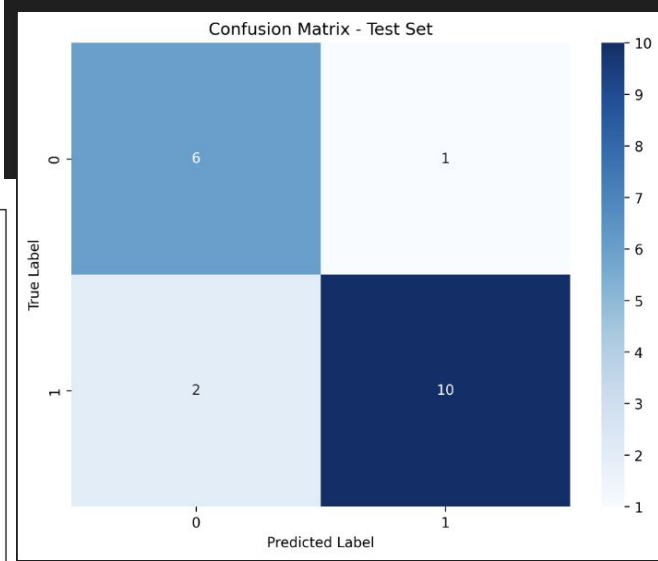
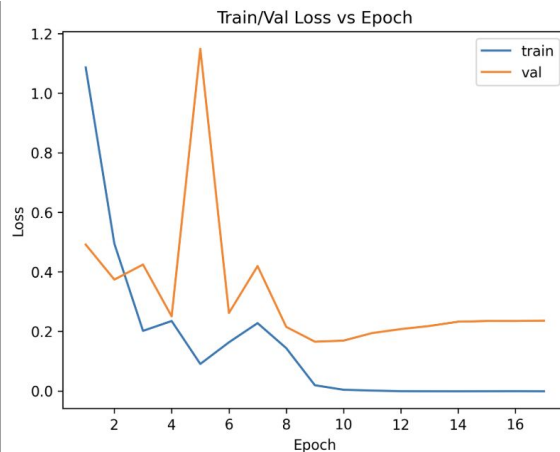
- New method: `_save_deviation_plot(results, output_dir)`
 - Computes: $\text{deviation} = P(\text{class}=1) - y_{\text{true}}$
 - Saves: `deviation_plot.png`
- `Evaluate()` now saves it
 - Old: saved `predictions.csv` + `confusion_matrix.png`
 - New: also saves `deviation_plot.png`
- Added a proper `save_loss_curve()` method to `MILTrainer`
 - Saves `loss_curve.png` (train loss + val loss vs epoch)
- `main.py` calls it after training
 - `trainer.save_loss_curve(run_dir)`
- Why:
 - These are explicit deliverables
 - Makes each run self-contained: you can open the run folder and see all metrics/plots immediately
 - Deviation plot adds more insight than accuracy alone (shows calibration + direction of error)

Main.py changes

- Call `trainer.save_loss_curve(run_dir)` after training
- Why:
 - Guarantees the loss plot is generated for every run
 - Keeps artifact generation centralized inside trainer so it's consistent across sweeps

All Benign Splits Results

- *Disclaimer: this version has early stopping set to 10 epochs, because my 30 epochs version is still running*
- Test Loss: 0.6516
- Test Acc: 0.8421
- Samples: 19
- High-grade recall: 0.833
- Benign recall: 0.857



All Benign Splits Results Con't

- Many cases showed highest attention on melan-stained patches
- Some cases relied more on sox10
- A few used H&E
- This suggests the model often relied on melan-stained regions, though attention varied by case.
- Note: Attention weights do not prove causal importance, but provide an indication of which regions influenced predictions.

Stain	Times most attended
Melan	13
Sox10	5
H&E	1

All Benign Splits Results Con't

Patch-level attention patterns

- Top 5% highest-attention patches per case
 - Many cases had top patches concentrated in a small number of slices
 - Ex: Case 73
 - Total patches: 1797
 - Top patches: 89
 - Slice distribution:
 - (melan, slice 0): 81
 - (melan, slice 2): 1
 - (sox10, slice 0): 7
 - In some cases, most high-attention patches were concentrated in a single slice of one stain.
 - In several cases, most high-attention patches came from a single stain
 - Ex: Case 55:
 - Top patches: 44
 - melan: 44
 - In this case, the highest-attention patches were all from melan-stained regions.
 - Some cases showed attention distributed across multiple stains
 - Ex: Case 70:
 - Top patches by stain:
 - h&e: 16
 - melan: 3
 - sox10: 25
 - High-attention patches came from multiple stains rather than one dominant stain.

Next steps

1. (use 30 epochs Early stopping limit to evaluate more)
 - a. Still currently running
2. Evaluate across multiple benign-family splits
 - a. Run the training pipeline while holding out each benign family as the test set
 - b. This ensures all benign family groups are evaluated without leakage
 - c. Comparing results across splits helps determine whether performance is consistent or dependent on a specific split
 - d. Why this matters:
 - i. A single test split (19 cases) may not fully represent dataset variability. Evaluating multiple splits provides a more robust estimate of model performance.

Optuna Optimization Studies

Implementation/Settings:

- Modify from Krish_Updates [Optuna_training_2.py](#), [Optuna_training_3.py](#), [optuna_sbatach.sbatch](#)
- 4 trials, 30 Epochs within each trial
- TPESampler→decides what values to try next.
- Pruning →stops unpromising trials
 - N_startup_trials = 4
 - Ensures that OPTUNA cannot prune any trial.
 - Pruning_warmup = 8
 - Within each trial OPTUNA cannot stop the trial early before the first 8 epochs.

Initial Parameters

Learning Weight	Weight Decay	Coefficient Dropout	Classifier Dropout	Class Weights	Entropy Coefficient
[1e-5,1e-3]	[1e-5,1e-2]	[0.05,0.20]	[0.05,0.20]	[1.5,3.5]	[1e-5,1e-2]

Studies

- Optimization (2 rounds)→ ER/GA, No DO, and No DO code

Reduced Model Performance due to changes to Attention Analysis Files

Files Changed: [attention_analysis.py](#), [dataset.py](#), and [data_utilz.py](#)

Effects

- **Reduced Run-Time:** [Pre Attention Analysis](#) → [Post Attention Analysis](#)
 - 4-5 hours → ~20 minutes
 - 500-800 seconds → 60-400 seconds.
- **Worse Performance: Mandatory_Krish_Changes**

	Accuracy	Loss	Benign Recall	High-Grade Recall
Pre-Attention	0.89	0.34	0.86	0.92
Post-Attention	0.74	0.38	0.57	0.83

- **Hyperparameter Optimization unsuccessful**
 - Accuracy → 0.78
 - Loss → 1.06

Next Steps

- Tune model with no attention changes to the code using OPTUNA optimization.
- Fix Attention Analysis Files

Akshay - Hyperparameter Tuning Strategy

Dimensionality of hyperparameter space

We want to optimize

- Learning rate
- Weight decay
- Dropout
- Class weight
- Scheduler minimum LR

A naive grid search of 10 values per parameter would be:

$$10^5 = 100000$$

Such a high number of configurations = completely infeasible tuning method

Training learning rate...

Mathematical Justification

$$l(W_t) - l(W^*) \leq \frac{\|W_0 - W^*\|^2}{2nt}$$

Where

$$l(W_t) = \text{loss @ iteration } t$$

$$l(W^*) = \text{loss @ iteration } t^{\text{OPTIMAL}}$$

$$W_0 = \text{initial weights}$$

$$W^* = \text{optimal weights}$$

$$n = \text{learning rate}$$

$$t = \# \text{ of iterations / epochs}$$

The equation above is essentially:

$$\text{error} \leq \frac{(\text{Initial distance})^2}{2nt}$$

$$e \leq \frac{\|W_0 - W^*\|^2}{2nt}$$

$$t \geq \frac{\|W_0 - W^*\|^2}{2ne}$$

$\therefore$

$$t \propto \frac{1}{n}$$

- Learning rate can be adapted/updated dynamically with [1] RMSProp, [2] Adam, and [3] ReduceLROnPlateau

Of these possible solutions, I believe that ReduceLROnPlateau would work best because:

- It can actively adjust step size based on how far the model is from convergence
- We are training on countless hyperparameter configurations so without adaptive scheduling we would have to train each epoch separately

Possible solutions

Both Random sampling and LHS will decrease computational cost compared to naive grid search, but

[1] Random Sampling in a 5 dimensional plane will

- Lead to possible clustering

[2] Latin Hypercube Sampling

- Force the sampling process to sample from each bin/section of the model

$$x_{i,k} = \frac{\pi_k(i) - u_{i,k}}{N}$$

1. $i \in \{1, \dots, N\}$ is the sample index.

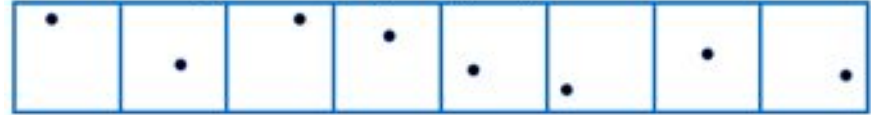
2. $\pi_k(\cdot)$ is a random permutation of $\{1, \dots, N\}$ for each dimension k .

3. $u_{i,k}$ is a uniform random number $U(0, 1)$.

(1) Simple random sampling



(2) Latin Hypercube sampling (LHS)



https://www.researchgate.net/figure/Simple-random-sampling-and-Latin-hypercube-sampling_fig3_390832064

Number of samples

Given that LHS provides us with samples from every section, how many total samples do we need to take?

- Typically 10x-20x trainable hyperparameters, but with the use of TPE and MP, only 3x are needed
- TPE models $p(\theta|y < y^*)$ and $p(\theta|y \geq y^*)$ and yields a few great and poor trials
- Given that we are training 5 parameters, we will have 15 starting points/number of samples
- If we consider below 20% and above 80% to be good/bad cutoffs, then

$$0.2 \times 15 = 3$$

Pruning: Exploration vs Exploitation

When do we stop training a certain epoch?

If median validation loss at epoch 3 is 0.52 and the current trial at epoch 3 is 0.61, the relative degradation is calculated to be:

$$(0.61 - 0.52) / 0.52 = 0.173 = 17.3\%.$$

If this value is larger than the average noise across trials, then pruning is justified

Safe Pruning Rule

Prune if:

$$L_{val}^{(trial)}(e) > \text{Median}(L_{val}^{(past)}(e)) \quad \text{for } e \geq 3$$

Pruning: Exploration vs Exploitation continued

Given that our update rule is:

$$\Delta w = \eta \nabla L$$

Our movement size is directly dependent on learning rate (η)

- We have to be careful of the case in which learning rate is small and the epoch has a bad relative degradation!

The difference between the low learning rate and high learning rate, is quantified by 100x. This indicates that convergence will occur at at 100 times a slower rate.

If:

$$\|\nabla L\| = 0.2$$

High LR:

$$\eta = 10^{-3}$$

$$\Delta w = 2 \times 10^{-4}$$

Low LR:

$$\eta = 10^{-5}$$

$$\Delta w = 2 \times 10^{-6}$$

Therefore, is it wrong to prune an epoch that is poor early but converges slower?

Steps for optimization

$$\text{Let : } T_1 = 30, \text{ \# of trials}$$

$$E_1 = 10, \text{ Epochs}$$

$$C_1 = 30 \times 10 = 300$$

$$\text{Let : } T_2 = 120$$

$$E_2 = 15 \text{ \& increased \# of epochs}$$

$$C_2 = 120 \times 15 = 1800$$

$$T_3 = 50$$

$$E_3 = 20$$

$$C_3 = 50 \times 20 = 1000$$

$$C_{\text{total}} = 300 + 1800 + 1000 = 3100$$

equal \# of epochs (20) @ each stage:

$$20 \cdot (30 + 120 + 50) = 4000$$

$$E_T \cdot (T_1 + T_2 + T_3)$$

$$\text{Savings are : } \frac{4000 - 3100}{4000} = 22.5\%$$

1:

- Run a coarse global search and use 15 initial points for LHS
- Train for 10 epochs

2:

- Narrow ranges using 20% cutoff
- Run 15 epochs
- Enable pruning

3:

- Reduce the search radius
- Focus on learning rate precision

When to stop?

2 cases in which we stop are

[1] If the average change in score per trial is less than the error/validation loss

[2] If the variance of the previous 10 values is approximately 0

$$\Delta_k = L_{k-1} - L_k \quad \text{where } L_k = \text{score of } w_k$$

If

$$\bar{\Delta} < \epsilon \quad \text{where } \epsilon = \text{error and}$$

$\bar{\Delta}$ is average change

then terminate tuning.

Also, if

$$\text{Var}(\theta \text{ previous } 10) \rightarrow 0$$

then the search has converged as well

Variable vs Fixed: Computational cost

For a given convolutional layer consisting of:

$$O(B * C * H * W * K^2)$$

Given that B is Batch normalization, C is channels, K is kernel size are all fixed based on architecture,

we can deduce that: $O(H * W)$

Therefore a fixed size complexity @ 224x224: $O(224^2)$

and a variable size complexity cost could be: $O((H * W)^2)$
With worst case being max R_i

The CNN computational cost can be broken down as follows:

- $O(B * C * H * W * K^2)$

Therefore:

- A fixed image size scales quadratically with resolution
- Variable large images can yield very high/infeasible computational values
- Capping the image sizes at 224x224 retains cost and predictability

Variable vs Fixed: ability to detect small pathological regions

Variable sizing would create consistency in the pixel clarity, and would not hurt the model's ability to find small pathological regions because the model is using Patch based MIL

Fixed sizing could create major issues in the pixel resolution of images. When upsizing, it is likely possible that the image will lose clarity.

Variable vs Fixed: Model Accuracy

- Fixed sizing provides a solution in which variance increases or decreases in a controlled manner. For instance, if the fixed size went up from 224x224 to 256x256, a controlled increase in variance would occur.
- Variable sizing on the other hand can have drastic effects on variance increase and the worst case scenario (dominated by high res outliers) can affect the accuracy of the model dramatically.

Should resolution be a hyperparameter?

No, resolution should not be a hyperparameter for 2 reasons:

[1] It would be a high impact hyperparameter: Model complexity and compute cost change quadratically with R (proven earlier)

and

[2] adding resolution as a hyperparam increases initial design size and affects training cost at a R^2 rate as well which makes every trial more expensive.

Recommended solution

With the analysis shown here, the final recommendation is as follows:

- If the image $< 224 \times 224$ -> retain original size
- If the image $\geq 224 \times 224$ -> resize to 224×224

Reasoning:

- This caps the computational complexity issues with large image outliers
- Maintains resolution and pixel clarity in smaller images
- Controls variance in the large images

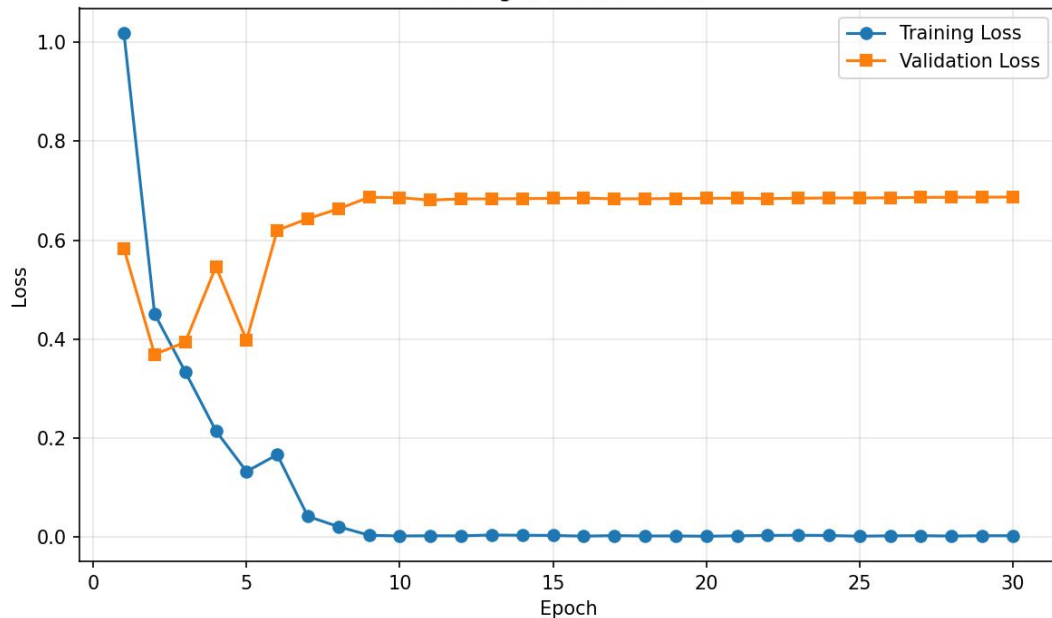
Best Model Results (Mandatory Task)

Gated Attention with Attention Analysis

(pres3_GA_AA_zoe)

Test Loss: 0.3181, Test Acc: 0.8421, Samples: 19

Training & Validation Loss

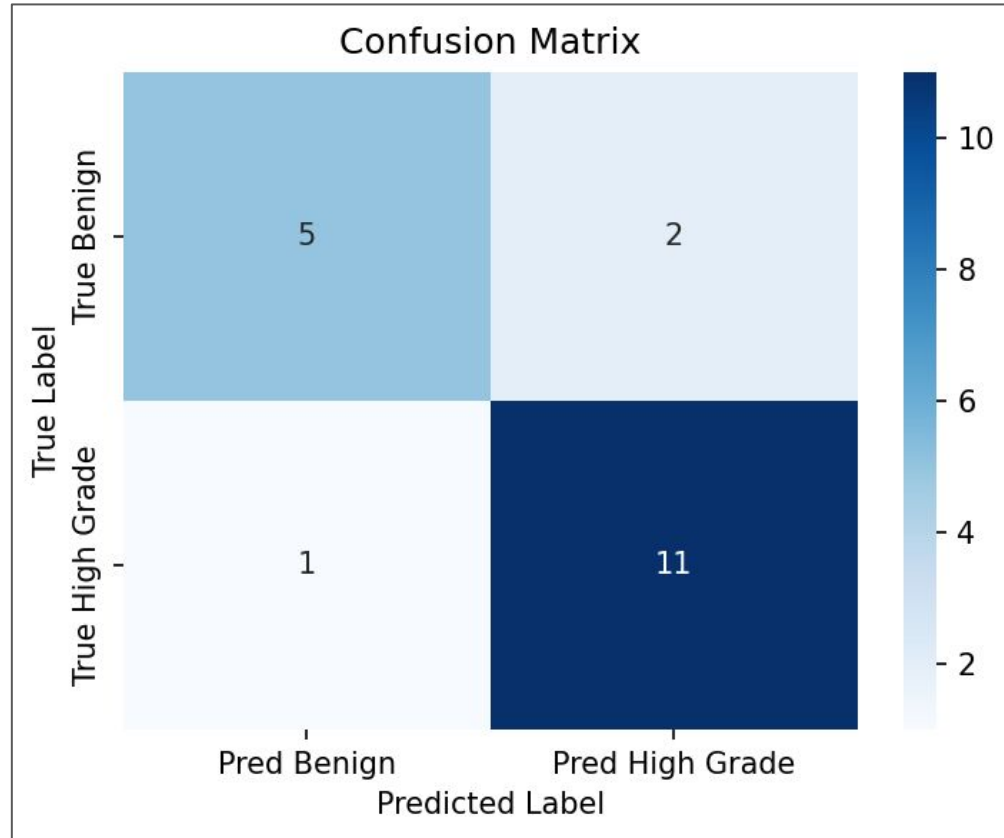


- Training loss falls ~ 0 rapidly
 - Model fits training data well
- Validation loss initially decreases
 - Model learns useful general patterns early
 - Best validation $\sim$ epoch 6
- Validation loss plateaus ~ 0.68 after epoch 10
 - Indicates loss of generalization ability
- Overfitting present

Github:

STAT390_Winter2026/Team4/Code
/Pres3_Results_Visualization_Bella.ipynb

Best Model Results (Mandatory Task)



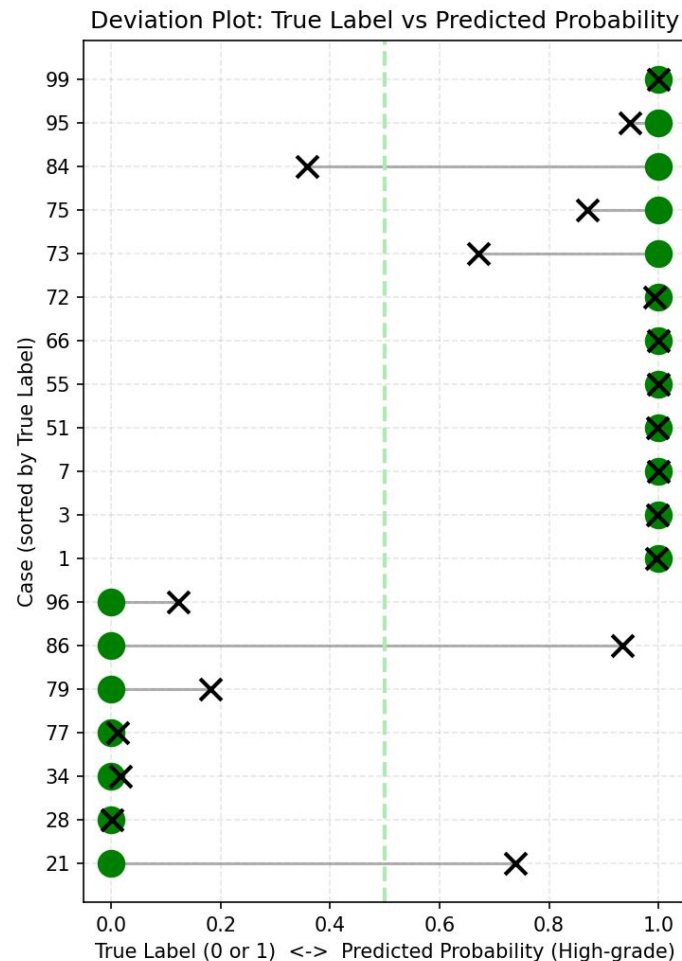
- TP=11, FP=3, TN=5, FN=1 (N=20)
- Good at identifying high-grade cases

Github:
STAT390_Winter2026/Team4/Code
/Pres3_Results_Visualization_Bella.ipynb

Best Model Results (Mandatory Task)

- Well-trained, not underfitting
 - Underfit models produce lots of predictions near 0.5
- Confident in most predictions, some uncertainties present
- 3 misclassifications: Cases 21 (B), 84 (HG), 86 (B)

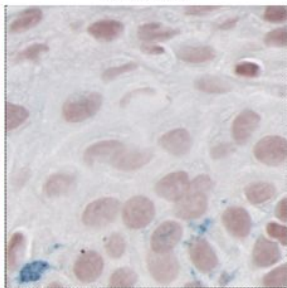
Github:
 STAT390_Winter2026/Team4/Code
 /Pres3_Results_Visualization_Bella.ipynb



Misclassified Benign Cases (Mandatory Task)

Case 21 - Top Effective Patches

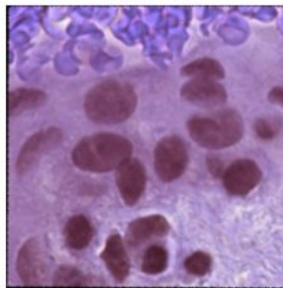
melan s0 p115
Eff: 0.3413



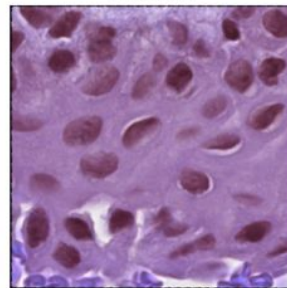
h&e s1 p48
Eff: 0.0530



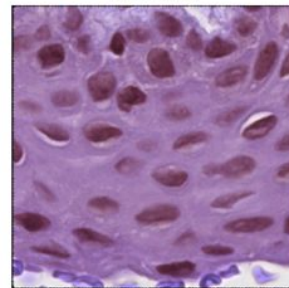
h&e s2 p176
Eff: 0.0487



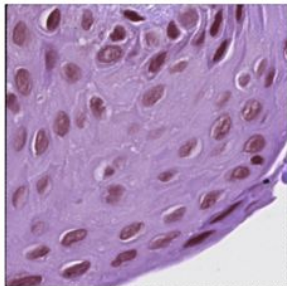
h&e s2 p175
Eff: 0.0424



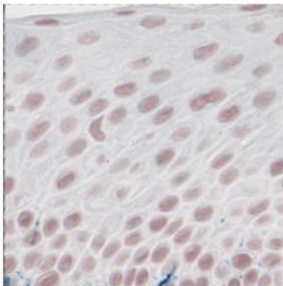
h&e s2 p174
Eff: 0.0420



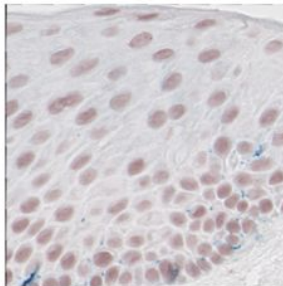
h&e s1 p49
Eff: 0.0299



melan s0 p82
Eff: 0.0290



melan s0 p81
Eff: 0.0285



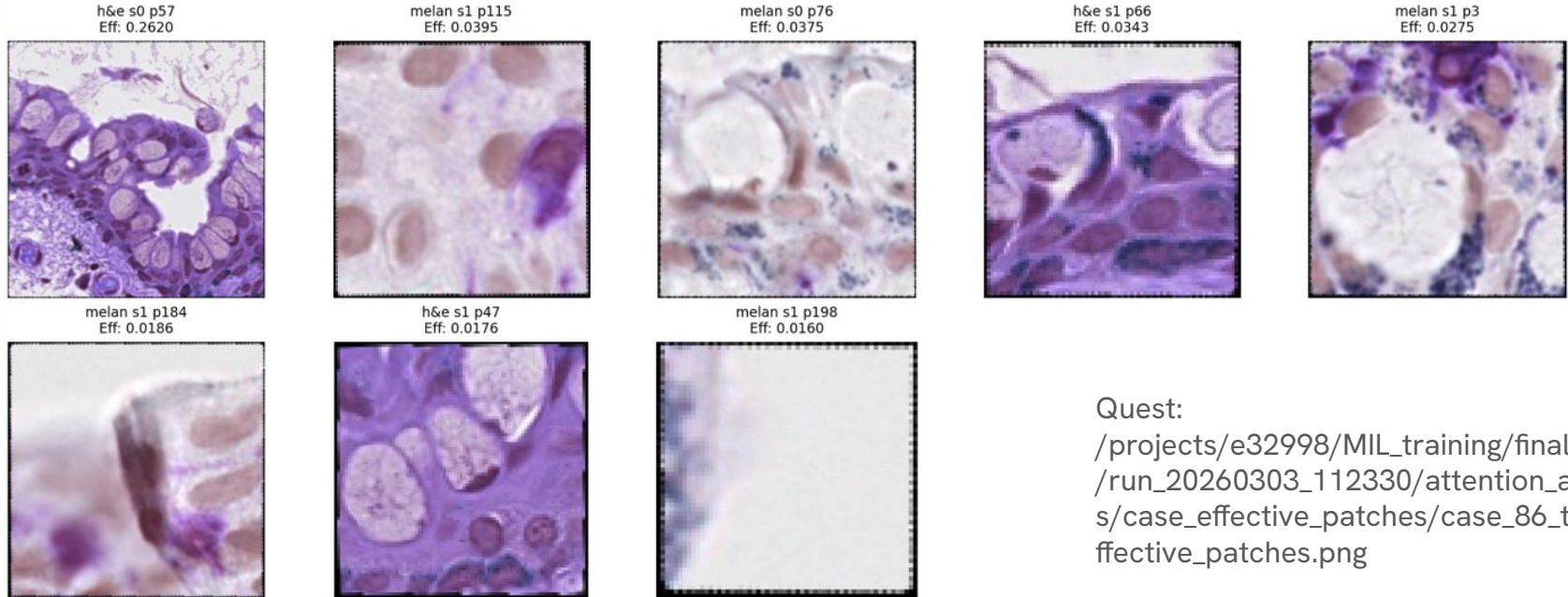
Quest:

/projects/e32998/MIL_training/final_runs/r
un_20260303_112330/attention_analysis/c
ase_effective_patches/case_21_top_effecti
ve_patches.png

Confident Misclassification, 1 patch dominates (prob_class0: 0.26135, prob_class1: 0.73865)

Misclassified Benign Cases (Mandatory Task)

Case 86 - Top Effective Patches



Quest:

/projects/e32998/MIL_training/final_runs
/run_20260303_112330/attention_analysis/
case_effective_patches/case_86_top_e
ffective_patches.png

VERY Confident Misclassification, 1 patch dominates (prob_class0: 0.06663, prob_class1: 0.93337)

How the Code Works- Preliminary

Purpose: Automatically extract square patches from tissue slices based on epithelium contours, weight them, and export images & metrics.

Setup:

- Imports OpenCV, NumPy, Shapely, Matplotlib, pandas, scipy.
- Parameters: patch overlap, step size, extension factor, smoothing sigma, max width.
- Input folder: testing_slices/ (contains slice images .tif and masks .png).

Outputs:

- Patch images (patch_outputs/)
- Metrics CSVs (patch_csvs/metrics_*.csv)
- Patch lengths per contour (patch_csvs/patch_length_*.csv)

Github:
STAT390_CMIL_Fall2025/Code/Patching
code/Patching_Code_Attention_Annika.ipynb

How the Code Works

Contour Analysis:

- `find_external_contours` → detects tissue contours from binary mask
- `estimate_curvature_complexity` → quantifies contour complexity via curvature peaks
- `gaussian_normals` → smooths contour/calculates normals for patch alignment

Patch Generation:

- `calculate_patch_corners` → defines square patch along normal, scaled by extension factor
- `create_smarter_squares` → generates non-overlapping, optimally sized patches along contours

Visualization:

- `draw_squares` / `draw_patches` → overlay patches on OG img/mask
- `show_shifted_patches` → handles padding and displays shifted patches alongside original mask

Patch Export:

- `extract_square_images` → crops/rotates patches to align with XY axes
- `save_patches` → saves individual patches to `patch_outputs/`.
- `export_patched_slice` → saves full slice overlay with patches.

Workflow Steps:

- 1) Identify matching mask & slice images
(`case_*_match_*_<stain>`).
- 2) Apply Otsu threshold to create epithelium binary mask.
- 3) Generate patch squares along contours.
- 4) Calculate metrics and handle padding/shift.
- 5) Visualize/export patches & metrics CSVs.

Changes Made to Code

Github:

STAT390_Winter2026/Team4/Code

/Pres3_Patching_Code_Attention_Bella.ipynb

Isabella

Problem!

```
Number of squares: 3412
Number of weights: 16
```

- Pipeline generated 3412 patches, weights_df only had 16 weights (Initial trial w/ case_21)
- Patching pipeline creates all possible patches, weights_df only has attention values for “important” patches (top/bottom attention values)
- Not every patch has corresponding attention... ERROR

Changes!

patching_export():

- Extract case, stain, and slice index from CSV
- Only selects and uses attention scores for patches of slice being assessed
- Sorts them by patch_idx to align with the order of patches returned by process_contours

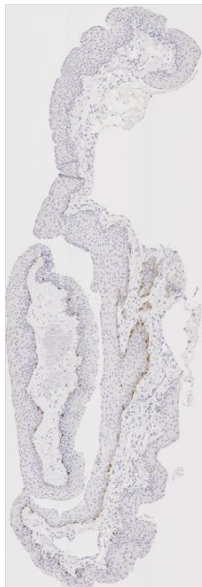
process_contours():

- Initializes weight array with zeros for all potential patches
- If patch has weight in CSV, assign attention scores from CSV to corresponding patch in visualization (use case, slice, and patch #)
- Patches without attention scores stay zero (no fill)

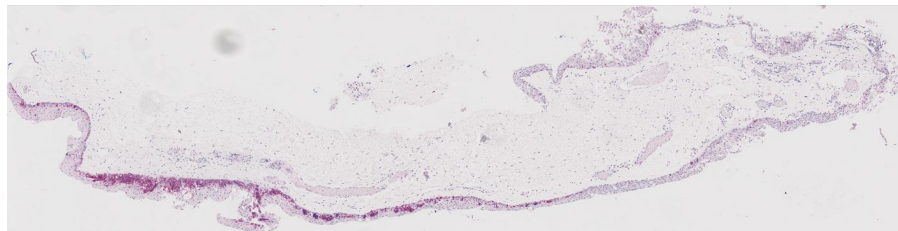
Simply put, match weights to patches on a per-slice basis using case, slice, and patch index

Cases/Images Used

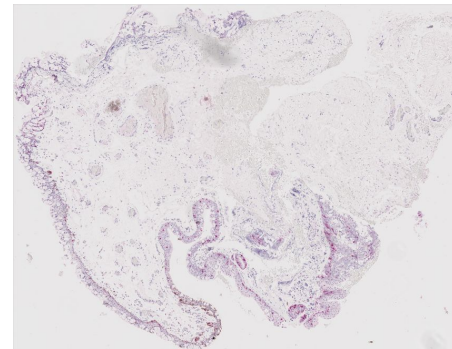
Based on results of the best model (pres3_GA_AA_zoe), 3 misclassified cases: 21(B), 84(HG), 86(B) **NOTE:**
All entries referring to Case 21 are missing from OneDrive [2. Annotated Slices (SP 2025)]



Case 21, Match 1, Melan
(PNG Unavailable)



Case 84, Match 1, Melan



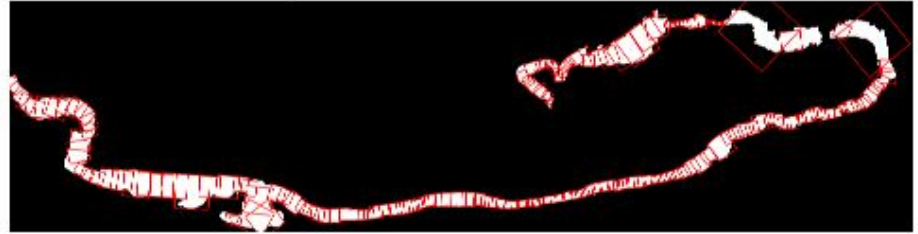
Case 86, Match 1, Melan

Direct Code Output: Case 84

Original Image



Grayscale Mask with Patches

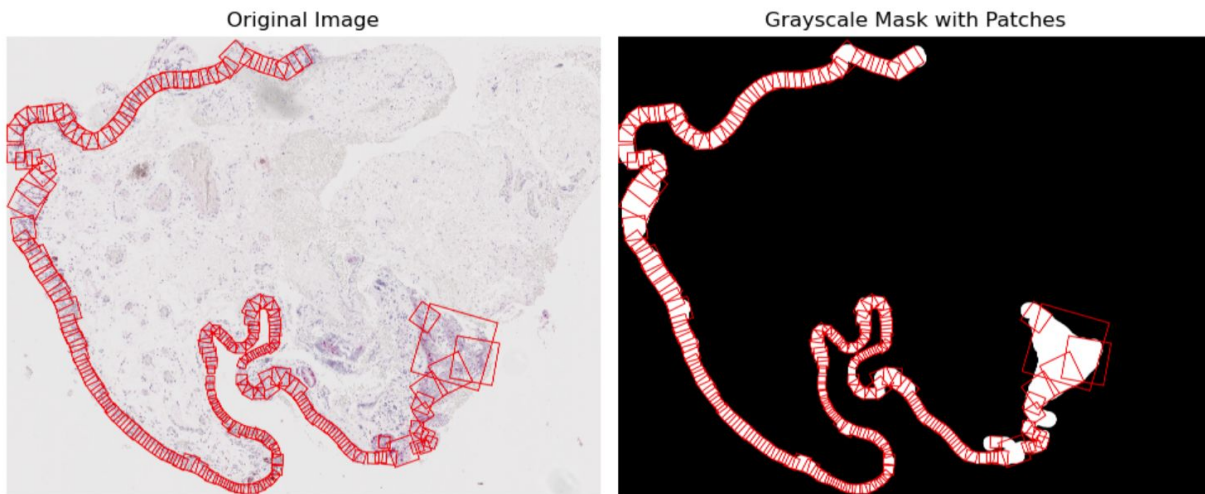


Github:
STAT390_Winter2026/Team4
/Code
/Pres3_Patching_Code_Attent
ion_Bella.ipynb

	Metric	Value
0	num_patches	176
1	total_epithelium_coverage	94.421775
2	background_pixel_percent	23.021955
3	score	90.933029
4	extension_factor	1.1
5	step_size	2
6	max_width	2048
7	overlap_threshold	0.3

✓ Patches
follow
epithelium!

Direct Code Output: Case 86

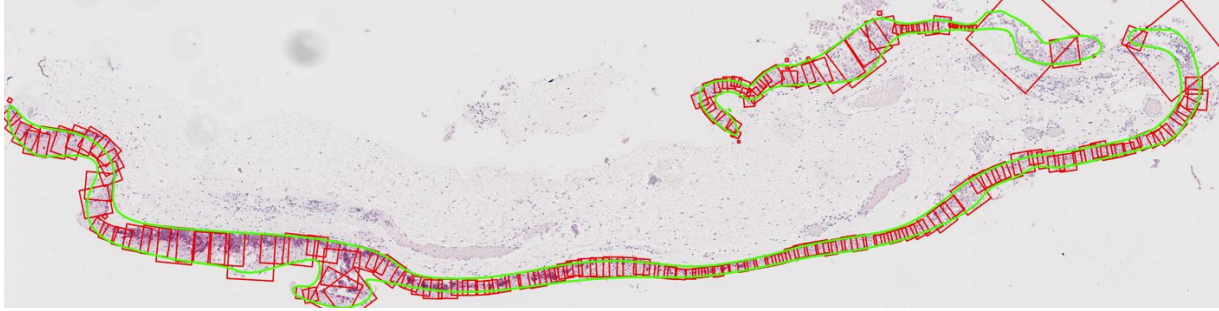


Github:
STAT390_Winter2026/Team4
/Code
/Pres3_Patching_Code_Attention_Bella.ipynb

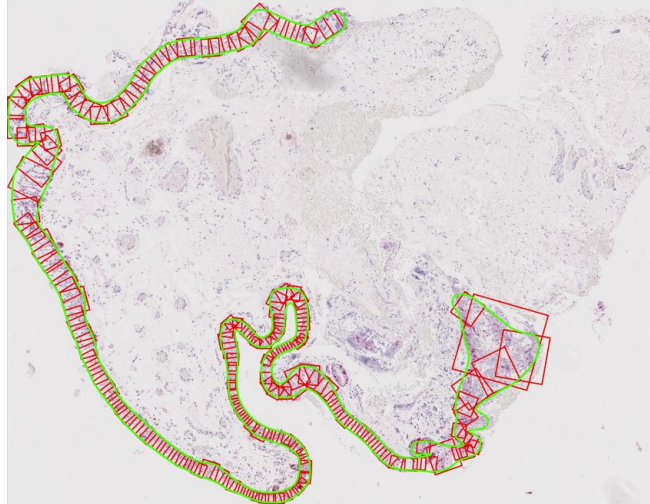
	Metric	Value
0	num_patches	178
1	total_epithelium_coverage	95.704344
2	background_pixel_percent	12.173922
3	score	94.128691
4	extension_factor	1.1
5	step_size	2
6	max_width	2048
7	overlap_threshold	0.3

✓ Patches
follow
epithelium!

Visualizations w/o Weights



Case 84

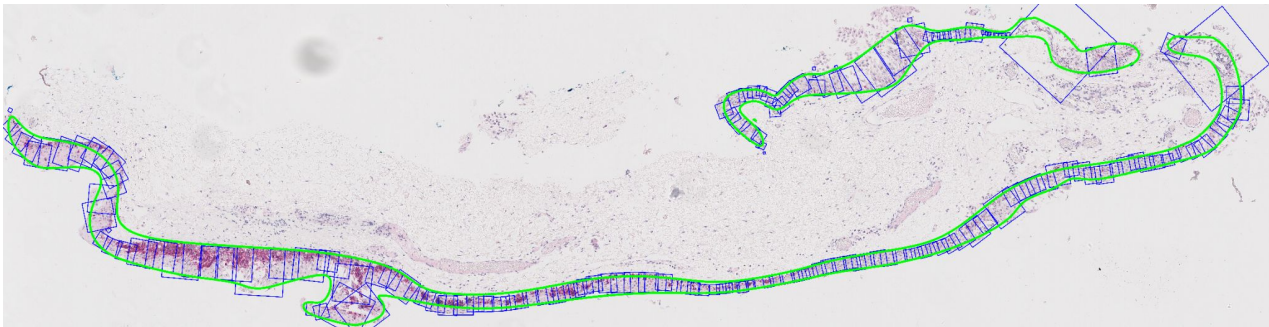


Case 86



Github:
STAT390_Winter2026/Team4/
Code
/Pres3_Patching_Code_Attenti
on_Bella.ipynb

Attention Considered, Final Output: Case 84



- Little/no fill because melan patches do not have high attention weights (as seen in table)
- All patches from Case 84, melan stain
 - Taken from “attention_analysis” CSV output of each run
 - top_effective_patches_per_case_5.0pct.csv
 - **RED** = highest attention weight across all other patches and stains for Case 84

Github:

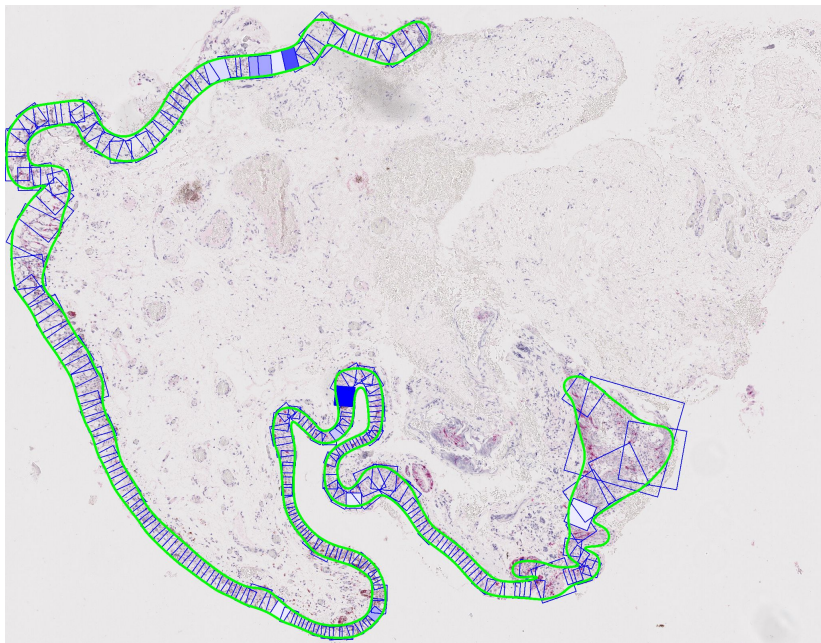
STAT390_Winter2026/Team4/Code
/Pres3_Patching_Code_Attention_Bella.ipynb

Case	Stain	Weighted Attention
84	melan	0.002981174
84	melan	0.003220398
84	melan	0.003349242
84	melan	0.003652399
84	melan	0.003660225
84	melan	0.003719665
84	melan	0.006545869
84	melan	0.006658866
84	melan	0.007937271
84	melan	0.008595999
84	melan	0.009217261
84	melan	0.018107189
84	melan	0.019455414
84	melan	0.025855875
84	melan	0.032253165
84	melan	0.03453923
84	melan	0.065737151

Attention Considered, Final Output: Case 86

- More fill because patches have high attention weights (as seen in table)
- All patches from Case 86, melan stain
 - Taken from "attention_analysis" CSV output of each run
 - Top_effective_patches_per_case_5.0pct.csv

Patch with strongest weight is h&e (included in table for reference), but melan holds 2nd place



Github:
 STAT390_Winter2026/Team4/Code
 /Pres3_Patching_Code_Attention_Bella.ipynb

Case	Stain	Weighted Attention
86	melan	0.002280286
86	melan	0.002510954
86	melan	0.002758988
86	melan	0.002760885
86	melan	0.003051498
86	melan	0.003192459
86	melan	0.003283968
86	melan	0.003465969
86	melan	0.003542404
86	melan	0.00416772
86	melan	0.004822065
86	melan	0.005034721
86	melan	0.005165549
86	melan	0.006042984
86	melan	0.007318175
86	melan	0.008309446
86	melan	0.008576451
86	melan	0.008858739
86	melan	0.012079357
86	melan	0.012959978
86	melan	0.012995032
86	melan	0.013341588
86	melan	0.013860147
86	melan	0.014373122
86	melan	0.016021178
86	melan	0.018575926
86	melan	0.027517945
86	melan	0.037515695
86	melan	0.039546996
86	h&e	0.261999204